

Universidad Católica Boliviana “San Pablo”

Facultad de Ingeniería

Carrera de Ingeniería de Sistemas

Segunda Evaluación de Tecnologías Web II 1-2025 – Primera Opción

Alina Mollinedo Dávila

1. Análisis del Proyecto Integrador

1) Identificación del endpoint

Endpoints a consumir:

- **GET** /api/historial-busqueda

Para obtener una lista de documentos que contengan una palabra clave ingresada

- Justificación de la elección del endpoint

Se eligió el endpoint /api/historial-busqueda

porque:

- Permite identificar documentos relacionados con temas específicos.
- Entrega resultados estructurados y consistentes, lo que simplifica la generación de métricas automáticas como conteo por año, jerarquía o vigencia.
- Facilita la reutilización del microservicio en distintos contextos, como tableros, reportes u otros sistemas que requieren análisis por término.
- En comparación con el endpoint de búsqueda por frase, este ofrece mayor control y precisión, evitando ambigüedades que surgen al dividir o interpretar frases completas.

- Descripción técnica del endpoint

Característica	Descripción
Método	GET
URL	/api/historial-busqueda
Formato de datos	String
Formato de respuesta	JSON
Autenticación	Ninguna

2. Diseño del Microservicio

- Objetivo del microservicio

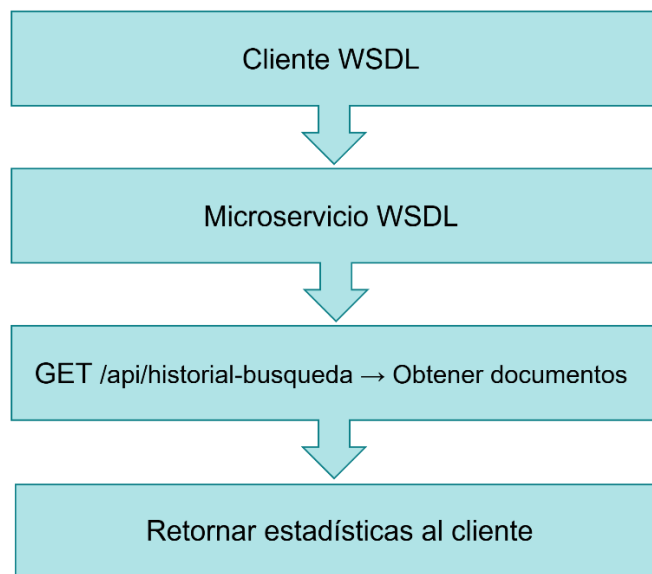
Objetivo: Construir un microservicio web WSDL en Express que permita a clientes consultar estadísticas acerca de un conjunto de documentos que contengan una palabra clave específica. Este consumirá la API y devolverá las estadísticas en formato SOAP.

- Elección y justificación de la tecnología de comunicación

WSDL (SOAP) fue elegido porque:

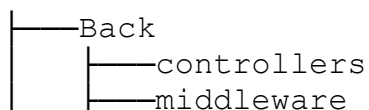
- Es ampliamente utilizado en sistemas gubernamentales y plataformas legadas, asegurando interoperabilidad.
- Permite describir claramente las operaciones, parámetros y estructuras de datos del microservicio.
- Utiliza esquemas XML que garantizan el cumplimiento del formato esperado.
- SOAP fue preferido por su nivel de formalismo y control sobre la definición del servicio.

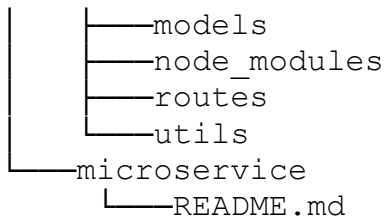
2) Diagrama del flujo de integración



3. Implementación Técnica

Estructura del proyecto





Código base

```
- wsdl-service.js
-
const express = require("express");
const soap = require("soap");
const axios = require("axios");
const app = express();
const port = 3001;

const service = {
  SearchHistoryService: {
    SearchHistoryPort: {
      getSearchHistoryStats: async function (args,
callback) {
        try {
          const keyword = args.keyword;
          const response = await
axios.get(`http://localhost:3000/api/historial-
busqueda?palabra=${encodeURIComponent(keyword)}`);
          const docs = response.data;
          const result = {
            total: 0,
            by_anio: {},
            by_jerarquia: {},
            by_vigente: {
              true: 0,
              false: 0
            }
          };
          if (docs.length === 0) {
            return callback(null, result);
          }

          result.total = docs.length;
          for (const doc of docs) {
            // Agregar por año
            if (!result.by_anio[doc.anio]) {
              result.by_anio[doc.anio] = 0;
            }
          }
        }
      }
    }
  }
};
```

```

        result.by_anio[doc.anio]++;

        // Agregar por jerarquía
        if
(!result.by_jerarquia[doc.jerarquia]) {
            result.by_jerarquia[doc.jerarquia
] = 0;
        }
        result.by_jerarquia[doc.jerarquia]+
+;

        // Agregar por vigente (true/false)
        const vigenteKey = doc.vigente ===
1 ? "true" : "false";
        result.by_vigente[vigenteKey]++;
    }

    return callback(null, result);
} catch (error) {
    console.error("Error al obtener los
resultados de búsqueda:", error);
    return callback(error);
}
},
},
},
};

const xml = `
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="SearchHistoryService"
    targetNamespace="http://www.example.com/docume
nts"
    xmlns="http://schemas.xmlsoap.org/wsdl/"
    xmlns:tns="http://www.example.com/documents"
    xmlns:soap="http://schemas.xmlsoap.org/wsdl/so
ap/"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema">

    <types>
        <xsd:schema
targetNamespace="http://www.example.com/documents">

            <xsd:element name="getSearchHistoryStats">
                <xsd:complexType>
                    <xsd:sequence>
                        <xsd:element name="keyword" type="xsd:string"/>

```

```

        </xsd:sequence>
    </xsd:complexType>
</xsd:element>

<xsd:element name="getDocumentStatsResponse">
    <xsd:complexType>
        <xsd:sequence>
            <xsd:element name="total" type="xsd:int"/>
            <xsd:element name="by_anio"
type="tns:StatEntryList"/>
            <xsd:element name="by_jerarquia"
type="tns:StatEntryList"/>
            <xsd:element name="by_vigente"
type="tns:StatEntryList"/>
        </xsd:sequence>
    </xsd:complexType>
</xsd:element>

<xsd:complexType name="StatEntryList">
    <xsd:sequence>
        <xsd:element name="entry" type="tns:StatEntry"
minOccurs="0" maxOccurs="unbounded"/>
    </xsd:sequence>
</xsd:complexType>

<xsd:complexType name="StatEntry">
    <xsd:sequence>
        <xsd:element name="key" type="xsd:string"/>
        <xsd:element name="value" type="xsd:int"/>
    </xsd:sequence>
</xsd:complexType>

</xsd:schema>
</types>

<message name="getSearchHistoryStatsRequest">
    <part name="parameters"
element="tns:getSearchHistoryStats"/>
</message>

<message name="getSearchHistoryStatsResponse">
    <part name="parameters"
element="tns:getDocumentStatsResponse"/>
</message>

<portType name="SearchHistoryPort">
    <operation name="getSearchHistoryStats">

```

```

        <input message="tns:getSearchHistoryStatsRequest"/>
        <output message="tns:getSearchHistoryStatsResponse"/>
    </operation>
</portType>

    <binding name="SearchHistoryBinding"
type="tns:SearchHistoryPort">
    <soap:binding style="document"
transport="http://schemas.xmlsoap.org/soap/http"/>
    <operation name="getSearchHistoryStats">
        <soap:operation soapAction="getSearchHistoryStats"/>
        <input>
            <soap:body use="literal"/>
        </input>
        <output>
            <soap:body use="literal"/>
        </output>
    </operation>
</binding>

    <service name="SearchHistoryService">
        <port name="SearchHistoryPort"
binding="tns:SearchHistoryBinding">
            <soap:address
location="http://localhost:${port}/wsdl"/>
        </port>
    </service>
</definitions>
`;

app.use(express.json());
app.listen(port, () => {
    console.log(`Servicio WSDL corriendo en
http://localhost:${port}/wsdl`);
});
soap.listen(app, "/wsdl", service, xml);

```

Librerías necesarias: npm install express soap axios

4. Pruebas y documentación

Capturas en postman

http://localhost:3001/wsdl

SaveShare

POST

http://localhost:3001/wsdl

Send

Params

Authorization

Headers (10)

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Beautify

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <SOAP-ENV:Envelope
3   xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
4   xmlns:tns="http://www.example.com/documents">
5
6   <SOAP-ENV:Header/>
7
8   <SOAP-ENV:Body>
9     <tns:getSearchHistoryStats>
10      <keyword>alimentacion</keyword>
11    </tns:getSearchHistoryStats>
12  </SOAP-ENV:Body>
13 </SOAP-ENV:Envelope>
14
```

Body

Cookies

Headers (6)

Test Results

200 OK · 46 ms · 654 B

XML

Preview

Visualize

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:tns="http://www.example.com/documents">
3   <soap:Body>
4     <tns:getDocumentStatsResponse>
5       <total>6</total>
6       <by_anio>
7         <2009>1</2009>
8         <2021>1</2021>
9         <2024>4</2024>
10      </by_anio>
11      <by_jerarquia>
12        <Suprema>1</Suprema>
13        <Media alta>5</Media alta>
14      </by_jerarquia>
15      <by_vigente>
16        <true>6</true>
17        <false>0</false>
18      </by_vigente>
19    </tns:getDocumentStatsResponse>
20  </soap:Body>
21 </soap:Envelope>
```